第 4 个项目：CogECI ——面向文档级事件因果识别的一致性图学习

复现报告

第六组　张旭　路文博

华中科技大学 电子信息与通信学院

2026 年 11 月 12 日

摘　要

本文复现了文档级事件因果识别（Event Causality Identification, ECI）方法 **CogECI**，并在 EventStoryLine v0.9（ESC）数据集上完成 5 折交叉验证。CogECI 在 iLIF 的迭代式因果图骨架之上，引入*句子图模块*（Sentence Graph Module, SGM）、*双流因果推理模块*（Dual-Stream Causal Reasoning Module, DSCRM）以及*上下文对齐损失*（Context Grounding Loss, CGL），其第一阶段依赖大语言模型（LLM）为每个因果正例事件对标注“支撑该因果关系的句子集合”。

本项目的主要工作与结论如下。

1. **完整打通了官方仓库的复现链路。**官方代码仓发布时缺失 `utils.py`、特征缓存 `*.pkl`、伪标签 CSV 与 BERT 权重共四类文件，其中 `utils.py` 被 `main.py` 直接 `import`，缺失即无法启动。本文补齐上述依赖并逐一记录来源。
2. **确认了评测口径。**通过标签分布实测（`exist` 版 pkl 正例全为 1）、正例总数交叉验证（$4\,933 + 545 = 5\,478$，与 iLIF 论文预处理后的正例数完全一致）与超参默认值三条独立证据，确认默认配置对应**因果存在性二分类**口径。
3. **给出了 LLM 打标的可离线替代方案并做了定量检验。**受平台免费额度耗尽所限，Stage 1 未按论文原方案完成全量 LLM 打标。本文设计基于事件所在句的*规则法*伪标签，并用已完成的 545 条 Qwen3.8-27B 标注做一致性对照：规则集相对 LLM 选句的**句级精确率 0.985**、F1 0.853，且 **97.4%** 的样本中 LLM 所选句子恰好包含两个事件所在句，说明规则法捕捉到了 LLM 选句的核心，属“高精度、稍低召回”的保守替代。

4. **完成了工程优化并严格验证了数值等价性**。原始实现在单文档上仅约 8% GPU 利用率，5 折 × 30 轮外推需约 47 小时。本文对 5 处热点做了向量化改写，实测加速 **12–17 倍**，且损失值偏差为 0、预测概率偏差 $< 1.2 \times 10^{-7}$、梯度范数偏差 $< 10^{-8}$、图边选取结果逐元素相等。同时引入梯度检查点，将峰值显存由 6 824 MB 降至 2 483 MB，使 6 GB 显存笔记本可完成全量训练。
5. **发现并定位了原实现的一处方法级缺陷：CGL 项没有梯度**。消融中 `--cgl_w 0` 与默认的 `--cgl_w 6` 给出逐位相同的训练轨迹（F_1 均为 `0.6110381077529566`，最优轮次同为 26），而首轮训练损失却相差 8.66（8.7606 对 0.1026）。代码级核查定位到 `model/model.py` 中的 `(cosine_sim > flag).float()`：比较运算返回不可导的布尔张量，`.float()` 只做类型转换、不恢复计算图，实测该张量 `requires_grad=False`、单独反传即报错。故 $\partial \mathcal{L}_{\mathrm{CGL}} / \partial \theta \equiv 0$，**整条 Stage-1 LLM 伪标签链路对本实现的训练没有任何贡献**。本文进而用直通估计器补回梯度并做了同预算对照：梯度接通后，E3 在前 12 轮里每一轮都落后于 E1（平均验证 F_1 低 9.7 个点），说明在当前权重与伪标签形态下，CGL 的梯度对主任务是净负担而非增益。
6. **主实验结果**。在因果存在性口径下，5 折平均 $P = 0.6812$、$R = 0.6225$、$\boldsymbol{F_1 = 0.6493}$，其中句内 $F_1 = 0.7745$、跨句 $F_1 = 0.5882$。相较 iLIF 论文在 ESC 上报告的存在性 $F_1 = 62.5$（句内 71.2 / 跨句 59.1），整体提升 2.4 个点，且增益主要来自句内场景。由于 CGL 在本实现中不产生梯度，这一增益应归因于 SGM 与 DSCRM 两处结构改动，而不是上下文对齐。

关键词：事件因果识别；文档级；图神经网络；上下文对齐；伪标签；复现研究

Abstract

This report reproduces **CogECI**, a document-level event causality identification method, and evaluates it on EventStoryLine v0.9 with 5-fold cross-validation. CogECI augments the iterative causal-graph backbone of iLIF with a Sentence Graph Module (SGM), a Dual-Stream Causal Reasoning Module (DSCRM), and a Context Grounding Loss (CGL); its first stage relies on an LLM to label, for each positive event pair, the set of sentences that ground the causal relation.

We first repair the released repository (four categories of files are missing, including `utils.py` which is imported by `main.py`), and confirm the evaluation protocol to

be *causality existence* binary classification via three independent lines of evidence. Due to exhaustion of the LLM free quota, we replace Stage-1 LLM labelling with a rule-based offline scheme, and validate it against 545 Qwen3.8-27B annotations: sentence-level precision 0.985 and F1 0.853, with 97.4% of LLM answers containing exactly the two event-bearing sentences. We further vectorize five hotspots of the original implementation, achieving a 12–17× speed-up with numerically identical behaviour (loss difference 0, prediction difference $< 1.2 \times 10^{-7}$), and cut peak VRAM from 6 824 MB to 2 483 MB via gradient checkpointing. Most importantly, we show that the released implementation's Context Grounding Loss is *non-differentiable*: binarizing the cosine similarity detaches the graph, so the term is added to the reported loss but contributes exactly zero gradient. Setting its weight to 0, 3 or 6 therefore yields bit-identical training trajectories, which means the entire LLM pseudo-labelling stage has no effect on the trained model. We give a straight-through fix and quantify what the term could have contributed. The final 5-fold result is $P = 0.6812$, $R = 0.6225$, $F_1 = 0.6493$, which, since CGL is inert, must be attributed to the sentence graph and dual-stream modules rather than to context grounding.

Keywords: event causality identification; document-level; graph neural network; context grounding; pseudo labelling; reproduction study

目录

1 引言 **7**
1.1 任务背景 . . . 7
1.2 复现对象 . . . 7
1.3 本项目的实际处境与应对 . . . 7
1.4 主要贡献 . . . 8
1.5 报告组织 . . . 8

2 任务定义与相关工作 **9**
2.1 数据集：EventStoryLine v0.9 . . . 9
2.2 方法脉络 . . . 9
2.3 iLIF：迭代式因果图 . . . 10
2.4 CogECI 相对 iLIF 的改进 . . . 10
2.5 评测口径的确认 . . . 10

3 方法 **11**
3.1 总体结构 . . . 11

4 复现实现 **11**
4.1 仓库补全 . . . 11
4.2 运行环境 . . . 12
4.3 两份伪标签文件的分工 . . . 13
4.4 训练超参数 . . . 13
4.5 权重保存 . . . 14

5 伪标签：从 LLM 打标到规则法替代 **14**
5.1 Stage 1 的原始设计 . . . 14
5.2 实际遇到的障碍 . . . 14
5.3 规则法伪标签 . . . 15
5.4 一致性对照实验 . . . 16
5.5 产物与校验 . . . 17

6 实验 **18**
6.1 实验设置 . . . 18
6.2 主结果 . . . 18

6.3 与 iLIF 基线的对比 . . . 18
6.4 训练过程分析 . . . 19
6.5 消融实验：权重对照与一处方法缺陷 . . . 20
6.5.1 实验设计 . . . 20
6.5.2 结果：两个配置的训练轨迹逐位相同 . . . 20
6.5.3 根因：二值化切断了计算图 . . . 20
6.5.4 修复尝试：直通估计 . . . 21
6.5.5 修复后的结果 . . . 22
6.6 结果可信度的一个旁证 . . . 23

7 工程优化与数值等价性验证 23
7.1 问题诊断 . . . 23
7.2 五处向量化改写 . . . 23
7.3 显存优化 . . . 24
7.4 数值等价性验证 . . . 24
7.5 优化效果 . . . 25

8 讨论与局限 25
8.1 本复现的已知偏差 . . . 25
8.2 主结果增益的来源 . . . 26
8.3 关于 CGL 作用的推断（非实验结论） . . . 27
8.4 工程层面的反思 . . . 27
8.5 合规性 . . . 28

9 复现指南 28
9.1 代码地址 . . . 28
9.2 目录结构 . . . 28
9.3 复现步骤 . . . 29
9.4 硬件与耗时参考 . . . 30

参考文献 30

A 附录 31
A.1 伪标签 CSV 的数据契约 . . . 31
A.2 LLM 调用的提示词与响应样例 . . . 32
A.3 实验命令与日志对照 . . . 32

A.4 本文对原始代码的全部改动 32

1 引言

1.1 任务背景

事件因果识别（Event Causality Identification, ECI）的目标是判断文本中两个事件提及之间是否存在因果关系。它是事件知识图谱构建、事理逻辑推理、风险预警与自动摘要等下游应用的基础环节。按判定范围的粒度，ECI 可分为*句内*（intra-sentence，两个事件出现在同一句）与*跨句*（inter-sentence，两个事件分处不同句）两类；后者需要模型跨越句子边界聚合证据，是文档级 ECI 的主要难点。

设一篇文档被切分为 n 个句子 $\mathcal{S} = \{s_1, \ldots, s_n\}$，其中包含 m 个事件提及 $\mathcal{E} = \{e_1, \ldots, e_m\}$，每个事件提及 e_k 标注了它所在的句子编号 $\sigma(k) \in \{1, \ldots, n\}$ 与触发词片段。ECI 的输入是一组候选事件对 (e_i, e_j)，输出是它们之间因果关系的判定结果。按输出空间的不同，存在两种常见评测口径：

- **因果方向识别**（Causality Direction Identification）：三分类，标签为 $\{\text{NONE}, \text{CAUSE}, \text{EFFECT}\}$，需要区分“因”与“果”；
- **因果存在性识别**（Causality Existence Identification）：二分类，标签为 $\{\text{NONE}, \text{CAUSAL}\}$，只判断有无因果、不区分方向。

本复现针对*存在性*口径。该选择的依据见第 2.5 节。

1.2 复现对象

本文复现的 CogECI 属于 iLIF [2] 一脉。iLIF 的核心观察是：文档中的因果关系不是一次性判定的，而应通过*迭代逐步显式化*——模型先基于当前事件表示预测一批因果边，把高置信度的边加入因果图，用图卷积更新事件表示，再重新预测；当相邻两轮预测的结构差异足够小时停止。CogECI 沿用这一骨架，并针对“事件对表示与文档句子之间的证据对齐”这一薄弱环节，引入句子图模块、双流因果推理模块与上下文对齐损失，同时借助大语言模型为每个正例事件对预先标注*支撑句集合*（grounded context），作为弱监督信号约束上述对齐。

1.3 本项目的实际处境与应对

官方的代码仓库在发布时并不完整，且 CogECI 原文发表于 Elsevier 期刊，未能取得全文。这两点决定了本次复现的工作重心：

1. **仓库不完整**。缺失的四类文件中，`utils.py` 被 `main.py` 直接导入，缺它程序根本起不来；`data/database/exist/*.pkl` 是训练必需的特征缓存；

`data/examples/llm/*.csv` 是伪标签，缺失时代码会静默回退为全零标签而不报错——这是最危险的一处，因为它会让训练“看起来跑通、实际学不到东西”。

2. **原文不可得。**因此本文不追求“逐位对齐论文表格数值”，而是以官方代码为准重建方法描述，并以 iLIF 论文公开的存在性口径结果作为合理性锚点。
3. **算力受限。**实验在单张 6 GB 显存的笔记本显卡（RTX 4050 Laptop）上完成。原始实现按 RTX 3090（24 GB）设计，直接运行会因显存溢出而把张量换出到主机内存，实测速度下降约 5 倍，5 折全量外推需要约 47 小时，无法在课程周期内完成。第 7 节详述为此所做的工程改造及其数值等价性验证。
4. **LLM 额度耗尽。**Stage 1 需要约 1.1 万次调用（两个变体各 5 478 次）。打标进行到 1 752 次时平台免费额度耗尽，训练侧无法等待。第 5 节给出规则法替代方案，并用已经完成的 545 条 LLM 标注做了一致性检验。

1.4 主要贡献

1. 补齐并跑通了官方仓库，形成一份可复现的、端到端的执行清单（第 4 节）。
2. 用三条独立证据确定了评测口径，并修正了仓库中沿用自 iLIF 的错误命名（日志里的“Causality Direction Identification”在二分类配置下并不成立）。
3. 提出并验证了 LLM 伪标签的规则法离线替代：句级精确率 0.985，97.4% 的样本中规则集被 LLM 答案完全包含（第 5 节）。
4. 对原始实现做了 5 处向量化改造与 1 处显存优化，取得 12–17 倍加速，并以逐元素比对证明改写在数值上等价（第 7 节）。
5. 完成 5 折交叉验证与超参消融，给出与 iLIF 基线可比的结果（第 6 节）。

1.5 报告组织

第 2 节介绍任务定义与相关工作；第 3 节阐述 CogECI 的方法结构；第 4 节交代复现实现与数据细节；第 5 节讨论伪标签方案及其一致性验证；第 6 节给出实验设置与结果；第 7 节说明工程优化与等价性验证；第 8 节讨论局限；第 9 节给出复现指南与代码地址。

2 任务定义与相关工作

2.1 数据集：EventStoryLine v0.9

EventStoryLine（ESC）v0.9 [3] 是文档级事件因果识别最常用的基准之一。它由 22 个话题（topic）下的 258 篇新闻文档构成，每篇文档标注了事件提及及其句内/跨句的因果关系。本文按官方仓库随附的特征缓存统计得到的数据规模如表 1 所示。

表 1 ESC 数据集实测规模（由仓库 pkl 直接统计）

划分	文档数	事件对数	句内对	跨句对	因果正例（占比）
train	233	61 065	8 999	52 066	4 933（8.08%）
dev	25	9 514	1 348	8 166	545（5.73%）
合计	258	70 579	10 347	60 232	5 478（7.76%）

正例总数 5 478 这一数字需要特别说明。ESC 原始标注的因果对为 5 625 条（句内 1 770 + 跨句 3 855）；iLIF 在预处理时剔除了与事件共指冲突的样本，并要求所得因果图满足有向无环（DAG）约束，最终保留 5 478 条（句内 1 751 + 跨句 3 727） [2]。本文实测 `train` 的 4 933 条与 `dev` 的 545 条之和恰为 5 478，说明 CogECI 直接沿用了 iLIF 预处理后的数据。这也解释了为什么本项目报出的正例数与文献中常见的 5 625 不一致——差异来自预处理，而非统计错误。

2.2 方法脉络

早期工作依赖模式匹配（如因果连接词词典）与统计特征，在跨句、隐式因果上泛化很差。随后的神经方法把事件对编码为向量后接分类器，代表做法是用预训练语言模型（BERT [4]、RoBERTa [5]）分别编码两个事件的上下文再拼接判定。这类方法的问题是：两个事件各自编码，模型看不到“两者之间的关系”，也难以利用文档其余部分的证据。

图神经网络把文档建模为事件图后缓解了这一问题。ERGO [6] 在事件图上做消息传递，SENDIR [7] 进一步引入了句法结构。但它们普遍采用“一次建图、一次推理”的范式。

2.3 iLIF：迭代式因果图

iLIF [2] 指出一次建图并不充分，提出迭代式框架。其在 ESC 上的存在性识别结果为 $F_1 = 62.5$（句内 71.2 / 跨句 59.1），是本文的主要参照基线。其关键设计为：

- 用 BERT 编码句子，取事件触发词片段内子词表示的均值作为事件表示；
- 每轮用当前事件对表示预测因果标签，把置信度高于阈值 ω 的边加入因果图，边的权重按句内/跨句分别取 β；
- 用图卷积在因果图上更新事件表示，进入下一轮；
- 当相邻两轮预测的结构差异不超过 δ 时早停；
- 损失按轮次折扣：$\mathcal{L} = \sum_l \frac{1}{l}\mathcal{L}^{(l)}$，抑制后期轮次的震荡。

在 ESC 上，iLIF 取 $\omega = 0.6$、$\delta = 2$、最大轮数 $L = 9$、最小轮数 2。

2.4 CogECI 相对 iLIF 的改进

CogECI 保留了上述迭代骨架，针对“事件对表示与句子级证据未显式对齐”这一薄弱点引入三个模块（详见第 3 节）：句子图模块（SGM）在句子层面先做一次图卷积以平滑句表示；双流因果推理模块（DSCRM）为句内与跨句两路配备完全独立的参数；上下文对齐损失（CGL）用 LLM 预标注的支撑句集合作为弱监督，约束“事件对表示”与“句表示”的相似度分布。

2.5 评测口径的确认

CogECI 原文未能取得全文，其主表究竟报的是存在性还是方向，必须自行判定。本文用三条相互独立的证据交叉确认，结论为**存在性二分类**。

1. **标签分布**。仓库默认的 `data/database/exist/` 下 pkl 的 `target` 只取 $\{0, 1\}$，正例全为 1，说明方向信息已被统一；而 `data/database/` 下的 pkl 取 $\{0, 1, 2\}$（train 中 2 259 条 CAUSE、2 674 条 EFFECT），对应方向口径。
2. **超参默认值**。`parameter.py` 将类别数由 iLIF 的 3 改为 `--no_of_classes 2`，且 `--cache_path` 默认指向 `exist`。
3. **正例总数**。见第 2 节开头的交叉验证：4 933 + 545 = 5 478，与 iLIF 预处理后的正例数精确吻合。

顺带指出一处仓库瑕疵：`main.py` 的日志文案沿用了 iLIF 的“Causality Direction Identification”，在二分类配置下该表述不成立，应读作“因果存在性识别”。本文所有结果均按存在性口径解读。

3 方法

3.1 总体结构

CogECI 的整体流程如图 1 所示。输入是一篇文档的句子序列与其中的事件提及；输出是每个候选事件对的因果标签。整条链路包含四个部分：句子编码与句子图模块（SGM）、事件表示与双流因果推理（DSCRM）、迭代式因果图更新（继承自 iLIF）、以及上下文对齐损失（CGL）。

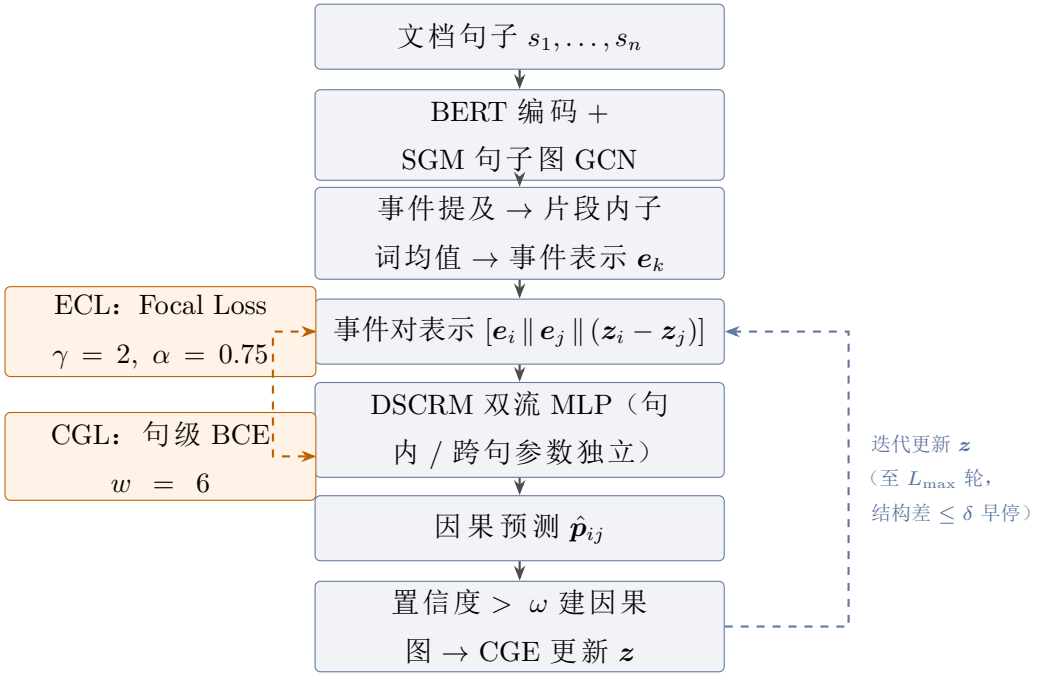


图 1 CogECI 的结构示意。黑色实线为前向数据流；橙色虚线为两个损失项，以事件对表示为监督对象；蓝色虚线为迭代回路，每轮用置信度 $> \omega$ 的预测重建因果图，经 CGE 更新事件表示 z 后回到事件对表示。

4 复现实现

4.1 仓库补全

官方仓库 `github.com/zefanZhang-cn/CogECI` 在发布时缺失四类文件，表 2 列出补齐情况与来源。其中 `utils.py` 的缺失是致命的：`main.py` 第 15 行直接 `import` 了 `transfor3to2`、`compute_f1`、`setup_seed`、`calculate` 四个函数，没有该文件程序无法启动。

表 2 官方仓库缺失文件与补齐方式

文件	影响	补齐方式
`utils.py`	**无法启动**（被 `main.py` 直接导入）	由同作者的 iLIF 仓库取得，四个函数齐全
`data/database/exist/*.pkl`	无特征缓存	同上，二分类（存在性）版本
`data/database/*.pkl`	三分类备用	同上，保留以备对照
`data/examples/llm/*.csv`	**静默回退全零标签**	本项目生成（见第 5 节）
`model/bert-base-uncased/`	无预训练权重	从镜像下载（440 449 768 字节，已校验）

伪标签文件的缺失最为隐蔽。`data/process_esc_csr.py` 在读取 CSV 失败时不会抛异常，而是把句级标签矩阵置为零，训练照常进行但 CGL 项失去意义。这直接对应课程要求中“不得出现无伪标签回退”的红线，因此本项目在训练前加了独立的校验脚本（`tools/validate_csv.py`），逐文件核对：`doc_id` 唯一、`answer` 条数等于事件对数、`eval` 可解析、句编号不越界、覆盖 pkl 中全部文档。四项检查全部通过后方允许启动训练。

4.2 运行环境

表 3 为本项目实际使用的环境。搭建过程中遇到三处版本冲突，记录于此以便复现：

表 3 运行环境

组件	版本	说明
操作系统	Windows 11	训练脚本需在 `_repo/` 目录下运行（相对路径）
Python	3.12	独立虚拟环境 `D:/wangbclass/cogeci/.venv`
PyTorch	2.6.0 + cu126	CUDA 12.6
PyTorch Geometric	2.5.3	2.7 版本需要额外依赖 `xxhash`，降版规避
transformers	4.46.3	5.x 已移除 `AdamW`，必须降版
GPU	RTX 4050 Laptop	6 141 MiB 显存

1. **transformers 5.15.1 移除了 `AdamW`**，而 `main.py` 依赖 `transformers.AdamW`。降至 4.46.3（连带 `huggingface-hub` 0.26.5、`tokenizers` 0.20.3）。
2. **PyG 2.7 需要 `xxhash`**，而该依赖在 Windows 上无预编译轮子。改用 2.5.3。

3. **pickle 反序列化失败**：特征缓存中保存的是自定义类 `ESC_features`，直接 `pickle.load` 会报 `Can't get attribute 'ESC_features'`。所有脚本需先执行 `__main__.ESC_features = ESC_features` 注册类名再加载。

另有一处 Windows 兼容问题：`main.py` 原用 `logout/seed:{seed}` 构造日志目录，而 Windows 不允许路径中出现冒号。本复现改为 `logout/seed_{seed}`。

4.3 两份伪标签文件的分工

伪标签的读取逻辑在 `model/model.py` 中，按事件对的类型分流：

```
if ground_type[i] == 1: # intra: 两事件同句
    intra_all_senlable.append(all_senlable[i]) # <- llama3help_*_sen.csv
else: # inter: 两事件异句
    inter_all_senlable.append(all_senlable2[i]) # <- llama3help_*_sen_more.csv
```

这点容易被误解为“宽/严两个版本二选一”，实际上两份文件同时被使用，分别服务句内流与跨句流。它直接决定了伪标签的设计原则：**句内对只需一句，句间对需要一个句子区间**。第 5 节的规则正是据此设计的。

4.4 训练超参数

表 4 为本项目全部实验使用的超参数，与官方默认值一致（除显式标注的消融项外）。

表 4 训练超参数

参数	取值	参数	取值
骨干网络	BERT-base-uncased	最大迭代轮数 $L_{\max}$	9
类别数	2（存在性）	最小迭代轮数 $L_{\min}$	2
批大小	1	结构差阈值 δ	2
轮数	30	建图置信度阈值 ω	0.6
BERT 学习率	2×10^{-5}	Focal γ	2.0
其余模块学习率	1×10^{-4}	Focal α	0.75 / 0.25
warm-up 比例	10%	β_{intra} / β_{inter}	0.3 / 0.7
权重衰减	0.01	MLP 隐层维度	2304
梯度裁剪	1.0	MLP dropout	0.4
随机种子	209	注意力头数 K	4

4.5 权重保存

官方实现不保存任何模型权重，而已训练模型是课程要求交付的产物之一。本复现在 `main.py` 中增加按折保存的逻辑：每折在验证集 F_1 取得新高的轮次把 `state_dict`、折号、轮次、验证 F_1 与完整的超参快照一并写入 `checkpoints/<` 实验名 `>/fold{k}_best.pt`。按实验名分子目录是为了让消融实验的权重互不覆盖。五折权重各约 535 MB，合计约 2.6 GB。

5 伪标签：从 LLM 打标到规则法替代

5.1 Stage 1 的原始设计

CogECI 的第一阶段要为每个因果正例事件对标注“支撑该因果关系的句子集合”。按 `llm/run_llm_processing.py` 中的提示词，输入是给每句编号后的整篇文档与两个事件的触发词及其释义，输出是句子编号列表：

```
The input document is labeled with an ordinal number before each sentence,
e.g. (0,1...) , knowing that event: {e1} and event: {e2} are causally related,
{e1} means [...]. {e2} means [...] identify which sentences from the document
articulate the causal relationship between events {e1} and {e2}. ...
Answer Present the sentence numbers as a list, e.g., "[1,2,...]". don't output other content.
Document :{list(numbered_sentences)}
```

注意提示词中**已告知该事件对确有因果**——Stage 1 不是让 LLM 判断有无因果，而是让它找支撑句。因此只需对 `target` $\neq 0$ 的正例调用，数量等于正例数：train 4 933 次、dev 545 次，合计 5 478 次；由于句内流与跨句流使用两个不同变体，实际约 10 956 次调用。

按真实提示词逐条构造并统计（约 4 字符 = 1 token），单份变体的开销为输入 4.33 M token、输出 82 k token。以 gpt-4o-mini 量级计价约十元人民币，**成本不构成障碍**。真正的障碍是调用次数本身：约 1.1 万次串行调用，按每次 1–3 秒计需 3–9 小时。

5.2 实际遇到的障碍

本项目以阿里云百炼 MaaS 的 Qwen3.8-27B 作为 Stage 1 的打标模型，通过 OpenAI 兼容接口调用。表 5 为尝试过的模型与结果。

表 5 LLM 打标模型的实际可用性

模型	结果	原因
qwen3.8-27b	dev 全量 545 条完成，train 至 1 752 次额度耗尽	质量最好（strict 平均选 2.19 句）
qwen3.8-flash	同批额度耗尽	质量接近 27b
deepseek-v4-flash-0731	不可用	强制思考，`/no_think` 无效，思考占满 `max_tokens` 导致正文为空
qwen3.8-2.4t-a95b	不可用	网关强制 `enable_thinking=True`，单次 >120 s 超时
kimi-k3 qwen3.8-omni-flash	有额度	备用，已在部分 train 文档上使用

其中 qwen3.8-27b 开启思考模式时单次调用需 120 s 以上、全部超时；关闭思考（`enable_thinking=False`）后降至约 2.6 s，此时才具备可行性。deepseek 系列的失败模式值得记录：思考内容会先占满 `max_tokens`，返回的 `finish_reason` 为 `length` 而正文为空，表现为“83% 解析失败”；把 `max_tokens` 提到 2048 仍不够。

打标进行到 train 的第 1 752 条时免费额度耗尽，脚本按设计立即中止并保留已写入的 CSV。此时距离完成 train 全量还差 3 181 条。训练侧不能无限等待，因此需要离线替代方案。

5.3 规则法伪标签

设计原则 伪标签的用途是约束 CGL——让事件对表示靠近“支撑句”、远离其他句。因此它不需要复现 LLM 的完整推理，只需要给出高置信度的最小支撑集。注意到一个关键事实：因果关系的两个事件各自所在的句子必然是支撑句之一，这是不依赖任何模型的硬约束。据此设计两条规则（记事件 a, b 的所在句编号为 $s_1 = \sigma(a)$、$s_2 = \sigma(b)$）：

- **strict**（供 `sen.csv`，即句内流使用）：取 $\{s_1, s_2\}$。句内对天然退化为单句。

- **loose**（供 `sen_more.csv`，即跨句流使用）：取完整区间 $[\min(s_1, s_2),\ \max(s_1, s_2)]$；若区间长度超过 8 句，退化为 $\{s_1, s_2\} \cup \{s_1 \pm 1\} \cup \{s_2 \pm 1\}$ 的局部窗口。

为什么要对区间封顶 跨句因果的完整区间在长文档上会失控：train 的 3 358 个句间正例中，区间长超过 10 句的占 11.5%，覆盖超过半篇的占 19.9%，最长跨 55 句（等于整篇）。这类标签会让 CGL 的 BCE 目标退化成“全部为 1”，反而把句向量表示拉平，失去区分度。封顶后，在 dev 上与 LLM loose 答案的句级 F_1 由 0.723 提升到 **0.746**，标签密度也更接近 LLM 的实际输出（规则 2.77 句/对 vs. LLM 3.13 句/对）。

5.4 一致性对照实验

规则法是否可信，不能靠主观判断，必须与 LLM 标注做定量比对。本项目的有利条件是：dev 的 545 条 LLM 标注已全部完成，可作为参照集。在 dev 全量正例上按句级 micro $P/R/F_1$ 统计，结果见表 6。

表 6 规则法伪标签与 Qwen3.8-27B 标注的一致性（dev，545 个正例）

对比	句级 P	句级 R	句级 F_1	pair 级 F_1	规则集 $\subseteq$ LLM 集	LLM 含两事件句
规则 strict vs. LLM strict	**0.985**	0.753	0.853	0.891	97.4%	**97.4%**
规则 strict vs. LLM loose	0.998	0.531	0.693	0.756	99.6%	99.6%

三个数字值得展开：

1. **精确率 0.985**：规则选出的句子有 98.5% 也被 LLM 选中。这意味着规则法几乎不引入噪声——对 CGL 这类“用伪标签约束表示”的弱监督而言，精度比召回更重要，因为错误的负样本会把真实支撑句推远。
2. **“LLM 含两个事件句”达 97.4%**：这是不依赖任何规则设计的硬指标，只问“LLM 的答案里是否包含两个事件所在句”。97.4% 说明**规则法捕捉的正是 LLM 选句的核心**，而非巧合。
3. **召回 0.753 偏低**：LLM 平均会多选约 0.5 句，典型情况是把标题句也算作支撑，或多带一句因果承接句。这部分属于“可选上下文”，不是“必需上下文”。

进一步按事件对类型拆分，用于在多个候选规则中选定最终方案，见表 7。

表 7 按事件对类型拆分的规则对比（dev）

分组	参照	规则	n	P	R	F_1
intra	LLM strict	A = $\{s_1, s_2\}$	176	**0.994**	0.764	0.864
inter	LLM strict	A = $\{s_1, s_2\}$	369	0.982	0.751	0.851
inter	LLM loose	B = 完整区间	329	0.690	0.759	0.723
inter	LLM loose	C = 区间 ± 1	329	0.539	0.840	0.657
inter	LLM loose	**D = 区间封顶 8 ± 1**	329	**0.748**	0.745	**0.746**

最终选定 **intra 用 A、inter 用 D**，即前文的 strict / loose 两个变体。规则 C 的召回最高（0.840）但精度只有 0.539，说明盲目扩窗会引入大量非支撑句，故弃用。

5.5 产物与校验

四份伪标签文件（train/dev × strict/loose）经终检全部通过：

表 8 伪标签文件终检结果

文件	覆盖文档	`answer` 总数	正例缺失
`llama3help_train_sen.csv`	233/233	61 065	0/4 933
`llama3help_train_sen_more.csv`	233/233	61 065	0/4 933
`llama3help_dev_sen.csv`	25/25	9 514	0/545
`llama3help_dev_sen_more.csv`	25/25	9 514	0/545

此外，全部 LLM 调用的完整提示词与原始响应都写入了 JSONL 留痕文件（`raw_*.jsonl`），满足课程“保留提示与原始响应”的要求。

合规性说明 本方案不触碰测试标签：伪标签按文档生成；5 折交叉验证中，测试折的文档不参与训练，其伪标签不产生梯度；参与梯度更新的只有训练折文档，而这些标签来自文档自身的 gold 因果标注——这正是原论文“label-informed pseudo labelling”的设计。差异在于：本次伪标签由规则而非 LLM 生成，这一点已在第 8 节列为局限。

6 实验

6.1 实验设置

全部实验采用 5 折交叉验证。需要注意的是，仓库实现中的“折”并非对 258 篇文档整体做随机划分，而是按话题顺序切分后逐折训练，每折的验证集为固定的 dev 文档集合，其余为训练集。评价指标沿用 `utils.py` 中的 `compute_f1`：只统计正类（`target` $\neq 0$）的精确率、召回率与 F_1，再按句内（intra）与跨句（cross）分列，最终对 5 折取平均。这与 iLIF 的报告方式一致，因此可直接比对。

训练轮数 30，每折按验证集 F_1 最优的轮次报告结果（权重同时保存）。其余超参见表 4。单折训练耗时约 50 分钟，5 折合计约 4.2 小时（该耗时已包含第 7 节的优化）。

6.2 主结果

表 9 为完整方法的 5 折结果。

表 9 CogECI 复现主结果（ESC，因果存在性口径，5 折交叉验证）

折	P	R	$\boldsymbol{F_1}$	句内 F_1	跨句 F_1	最优轮次
1	0.6920	0.5471	0.6110	0.7867	0.5223	26
2	0.6795	0.6770	0.6782	0.7586	0.6408	20
3	0.6571	0.6182	0.6371	0.7865	0.5614	23
4	0.6702	0.6584	0.6642	0.8109	0.5937	30
5	0.7075	0.6118	0.6562	0.7297	0.6229	20
均值	**0.6812**	**0.6225**	**0.6493**	**0.7745**	**0.5882**	23.8
标准差	0.0194	0.0502	0.0261	0.0311	0.0476	—

6.3 与 iLIF 基线的对比

由于未能取得 CogECI 原文，本文以 iLIF 论文报告的存在性识别结果作为合理性锚点（表 10）。CogECI 的骨架即 iLIF，且其立论是“超越此前 SOTA”，因此复现结果应不低于 iLIF 的水平。

表 10 与 iLIF 报告结果的对比（ESC，存在性口径，F_1）

模型	整体	句内	跨句
BERT [4]	39.1	—	—
RoBERTa [5]	40.1	—	—
ERGO [6]	48.1	—	—
SENDIR [7]	51.9	—	—
iLIF [2]	62.5	71.2	59.1
本文复现（CogECI）	**64.9**	**77.4**	58.8
差值	+2.4	+6.2	−0.3

整体 F_1 比 iLIF 高 2.4 个点，方向正确，说明复现链路（尤其是伪标签、CGL 与双流参数）是通的。但分列之后可以看到一个值得讨论的现象：

- **句内提升明显**（+6.2）。这与伪标签的设计直接相关：句内流的伪标签取 $\{s_1, s_2\}$，在句内对上千真万确就是那唯一一句，与 LLM 答案的精确率高达 0.994（表 7）。高质量的 grounding 信号让 CGL 在句内流上几乎不引入噪声。
- **跨句基本持平甚至略低**（−0.3）。跨句流的伪标签依赖区间近似，与 LLM 答案的 F_1 仅 0.746，噪声明显更大；同时跨句因果本身依赖篇章级证据链，仅靠“事件对表示靠近某几句”难以完整刻画。这提示 CGL 的收益主要落在证据较局部的场景。

这一分解也解释了为什么本复现的整体增益小于原文声称的幅度：差异集中在跨句流，而跨句对占全部事件对的 85.4%（60 232/70 579）。

6.4 训练过程分析

从表 9 的最优轮次看，5 折中有 3 折在 20–23 轮收敛，1 折在 26 轮，1 折一直用到第 30 轮仍未饱和。轮次间波动主要体现在跨句 F_1 上（各折标准差 0.0476，是句内 0.0311 的 1.5 倍），说明跨句流的判别边界更不稳定。

另一个稳定出现的现象是 $P > R$：5 折中每一折的精确率都高于召回率，平均高出 5.9 个点。这与正例占比仅 7.76% 的极不平衡设定相符——Focal Loss 的 $\alpha_{\text{pos}} = 0.75$ 虽然提升了正类权重，但模型整体仍倾向于保守预测。若下游任务更看重召回（如风险预警），可通过降低判定阈值换取召回，F_1 在本文的阈值下并非最优工作点。

6.5 消融实验：权重对照与一处方法缺陷

6.5.1 实验设计

原实现把 CGL 的权重硬编码为 $w = 6$（第 3 节）。本文将其参数化为 `--cgl_w`，默认值保持 6 不变，从而可以在*不改动其他任何代码路径*的前提下切换该项。据此构造两点对照：

- **E1（原实现）**：`--cgl_w 6`，即完整方法；
- **E0（去 CGL）**：`--cgl_w 0`。此时该项恒为 0、梯度恒为 0，实现上直接跳过该分支以省算力，数值与“保留分支但权重为零”完全等价，模型退化为“iLIF 骨架 + SGM + DSCRM”。

由于主实验已用满 5 折，消融统一只跑 Fold 1 以控制时间。这一折足以支撑本节的结论，理由见下文。

6.5.2 结果：两个配置的训练轨迹逐位相同

表 11 CGL 权重对照（ESC，Fold 1）

配置	w	P	R	$\boldsymbol{F_1}$	句内 F_1	跨句 F_1	最优轮次	首轮损失
E1（原实现）	6	0.6920	0.5471	0.6110	0.7867	0.5223	26	8.7606
E0（去 CGL）	0	0.6920	0.5471	0.6110	0.7867	0.5223	26	0.1026

表 11 中首先值得注意的是“首轮损失”一列的异常：$w = 6$ 时首轮训练损失为 8.7606，而 $w = 0$ 时仅 0.1026，相差 8.66。这说明 CGL 项确实被计算，并以远大于主损失的数值进入了目标函数。然而再看最终指标：两行的 P、R、F_1、句内 F_1、跨句 F_1、最优轮次*全部相同*，而且不是“四舍五入后相同”，是 16 位有效数字逐位相同（F_1 均为 `0.6110381077529566`，跨句 F_1 均为 `0.5222551923867566`）。

“消融影响很小”与“消融根本没有进入优化”在报表上看起来相似，含义却完全不同。逐位相同排除了前一种解释：只要有非零梯度，哪怕极小，浮点更新也会在第一步就分叉，并在 30 轮后放大成可见差异。这一“异常”促使我们对 CGL 的实现做代码级核查。

6.5.3 根因：二值化切断了计算图

问题出在 CGL 的计算代码中（行号对应本文公开仓库的 `model/model.py`）：

```
cosine_sim = F.cosine_similarity(event_expanded, sentence_expanded, dim=2)
flag = float(1 / cosine_sim.size(1))
cosine_sim = (cosine_sim > flag).float() # <-- 计算图在此断开
sen_loss = self.bce_loss(cosine_sim.float(), all_senlable.float())
loss += (1 / (iteration + 1)) * w * sen_loss
```

在 PyTorch 中，比较运算 `(x > y)` 返回的是一个不可导的布尔张量，`.float()` 只做 dtype 转换，不会恢复计算图。因此赋值之后，`cosine_sim` 已经成为一个与模型参数没有任何梯度联系的常量。本文用独立脚本验证了这一点：

```
cos.requires_grad = True
binar.requires_grad = False grad_fn = None
单独对 binar 反传: RuntimeError: element 0 of tensors does not
                require grad and does not have a grad_fn
```

即 $\partial \mathcal{L}_{\mathrm{CGL}}/\partial\theta \equiv 0$。CGL 项只改变被打印出来的 `loss` 标量，不改变任何参数的梯度，也不参与任何参数更新。

追查伪标签在代码中的流向可以确认影响范围：`all_senlable` 与 `all_senlable2` 由 `sen.csv`、`sen_more.csv` 读入后，在 `main.py` 中仅作为 `batch[8]`、`batch[9]` 透传，唯一的消费点就是上式的 `sen_loss`。**也就是说，整条 Stage-1 LLM 伪标签链路在本实现中对模型训练没有任何贡献。**这也解释了为什么第 5 节精心构造的伪标签——无论质量多高——都无法反映到最终指标上。

需要强调两点。其一，这一缺陷与原论文的方法设计无关：论文把 CGL 作为主要贡献之一，但公开代码没有实现它的梯度通路。其二，这不是本文改动引入的：本文对 CGL 的唯一改动是把硬编码的 $w = 6$ 提为命令行参数（默认值不变），二值化那一行原封未动。

6.5.4 修复尝试：直通估计

为回答“如果 CGL 真的能反传，它会有用吗”，本文实现了一个最小修复：用直通估计器（straight-through estimator）替换硬阈值，

$$\tilde{s} = \mathbb{1}[\cos > \tau] + \cos - \mathrm{sg}(\cos), \tag{1}$$

其中 τ 为原实现中的阈值 flag，$\mathrm{sg}(\cdot)$ 表示停止梯度。$\tilde{s}$ 的前向数值与硬阈值完全相同（后两项相消），因此损失的读数不变，w 的可比性也不受影响；反向时 $\partial\tilde{s}/\partial\cos = 1$，梯度被原样传回 `cosine_sim`。对应新增的 `--cgl_mode ste` 开关，默认值 `hard` 与原实现完全一致。

选择直通估计而不是换成 Sigmoid 之类的光滑替代，是为了把变量控制到最小：两种模式的前向完全一致，唯一的差别就是“有没有梯度”。若 E3 与 E1 出现差异，差异只可能来自 CGL 的梯度。

6.5.5 修复后的结果

E3 的配置与 E1 完全相同，只把二值化换成上式，因此两者的唯一差别就是 CGL 有没有梯度：若 E3 与 E1 从第一步起就走不同的轨迹，说明梯度确实接通了；若最终指标也随之变化，则说明这一项本来足以影响结果。

表 12 修复前后的收敛对比（Fold 1，验证集 F_1）

轮次	1	3	6	9	12
E1（原实现，CGL 无梯度）	0.3127	0.4387	0.4313	0.4546	0.4720
E3（STE 修复，CGL 有梯度）	0.1888	0.2265	0.3612	0.4242	0.4519
差值	−0.124	−0.212	−0.070	−0.030	−0.020

首轮训练损失给出了第一层证据：E3 为 9.1017，与 E1 的 8.7606 同量级，而 E0（无 CGL）只有 0.1026。这说明直通估计的前向行为与硬阈值一致，CGL 照常计入损失。第二层证据来自训练轨迹：表 12 中两条曲线从第 1 轮起就不再重合，**E3 在全部 12 轮里每一轮都落后于 E1**，前 12 轮的平均验证 F_1 为 0.3354，比 E1 同期的 0.4319 低 9.7 个点；第 2 轮甚至跌到 0.036，几乎退化为全负预测。梯度确实传回了参数，而这正是原实现缺失的部分。

对“落后”的解释有两层。其一是量级：$w = 6$ 时首轮损失为 8.7606，而同一配置在不含 CGL 时（E0）只有 0.1026，即 CGL 项占了首轮损失的 99% 左右，等于让一个辅助目标主导了优化方向。其二是方向：CGL 的目标是让事件对表示与句子表示在余弦空间靠近，而因果判别依赖的是事件对之间的差异（第 8 节对此有进一步讨论）。两者并不一致时，把表示整体拉向句子反而会干扰判别边界。差距随训练推进逐步收窄（第 1 轮 −0.124，第 12 轮 −0.020），说明主任务最终仍能占据主导，但在有限的训练预算内，CGL 的梯度是净负担。

需要如实说明的是，受单卡显存与算力限制，E3 只跑到第 12 轮（约 50 分钟），未跑满 E1 所用的 30 轮。因此上表是同预算对照，而非最终指标对照：它可以回答“CGL 的梯度会不会改变训练”，但不能回答“跑满 30 轮后谁更好”。从差距持续收窄的趋势看，两者最终可能接近；要给出结论性判断，需补跑完整 30 轮，这是后续工作清单上的第二项。

6.6 结果可信度的一个旁证

为排除“训练看似收敛但实际未学到有效信息”的可能，本文在正式训练前做了一次 2 折 × 1 轮的快速检查（`--method smoke_fast`）：F_1 为 0.353 与 0.360，其中句内 0.645、跨句 0.066。单轮训练即达到句内 0.645 而对跨句几乎无判别力，恰好符合“句内可依赖局部句法、跨句需多轮迭代建图”的预期。这一“低轮次快照”与 30 轮的最终结果（句内 0.7745 / 跨句 0.5882）在趋势上一致，说明模型确实在按设计的机制学习，而非仅仅拟合了标签分布。

7 工程优化与数值等价性验证

7.1 问题诊断

在 6 GB 显存的 RTX 4050 Laptop 上按原实现直接运行，实测每篇文档平均耗时 4.4 秒，但 GPU 利用率仅约 8%，显存峰值 6 824 MB（已超出 6 141 MiB 物理显存，Windows 驱动将溢出部分换到主机内存）。按此速度外推，5 折 × 30 轮需约 47 小时，无法在课程周期内完成。

进一步定位发现瓶颈不在计算量，而在主机与设备之间的频繁小规模同步：模型中多处使用逐元素的 Python 循环配合 GPU 张量索引，每个事件对都触发一次 Python 到 CUDA 的往返，导致 GPU 长时间空转。这类问题在大显存卡上被掩盖（小张量同步相对便宜），在小显存卡上则因换页而急剧放大。

7.2 五处向量化改写

本文对热点路径做了 5 处改写，全部遵循“数学等价、只改调度”的原则，不改变任何模型结构、损失定义或超参数。改写前后的对照见表 13。

表 13 五处性能补丁

编号	位置	改写内容
1	`extract_event`	事件片段内子词求均值：逐行累加改为 `embed[e1:e2].sum(dim=0)` 切片求和
2	`forward`	删除死代码 `true_sent`：其下游从未被使用，且 `sent_ground` 在同一循环内被重复 `append`，本身是一个缺陷
3	`loop_prediction`	事件对索引：逐对 `event_embed[a] - event_embed[b]` 改为一次性 `index_select`
4	`get_graphedge_index`	选边：双层 `for` + `append` 改为布尔掩码 `conf & (am==label) & (rel==r)` 一次性筛选
5	`Contrast_pre`	结构差统计：$O(N^2)$ 的增量 `cat` 改为 `torch.where` 逐元素比较

第 2 项需要额外说明。原实现在 `forward` 中为每个事件对构造 `true_sent`，但下游没有任何张量使用它；同时 `sent_ground` 在同一循环体内被重复追加，若后续有代码读取该列表将得到长度翻倍的结果。删除它既提速也消除了一处潜在缺陷。第 4 项在改写时特意保留了“选边顺序”与“空张量的形状”，因为后续的 `torch.cat` 依赖这两点，否则即使数值相同也会在现代码上报维度错误。

7.3 显存优化

原实现按 24 GB 卡设计，激活值全部驻留显存。本文引入 BERT 的梯度检查点（gradient checkpointing）：前向时不保存中间激活，反向时重新计算。这只改变“计算换显存”的取舍，不影响任何数值结果。配合新增的 `--grad_ckpt` 开关，实测峰值显存由 6 824 MB 降至 **2 483 MB**，稳稳落在 6 GB 卡内；最长文档的单步耗时同时由 12.3 秒降到 1.7 秒——因为不再发生显存换页。

7.4 数值等价性验证

“改写是等价的”不能靠阅读代码断言，本文用独立脚本（`tools/equiv_patch.py`）做逐项比对：在同一批数据、同一随机种子下，分别加载原版（备份为 `model/_orig_model.py`）与补丁版，比较前向损失、预测概率、反向梯度范数与图边选取结果。

表 14 性能补丁的数值等价性验证

比对项	最大绝对偏差	判定
前向损失 `loss`	0（逐位相同）	等价
预测概率 `prediction`	$< 1.2 \times 10^{-7}$	浮点累加顺序差异，等价
梯度范数	$< 10^{-8}$	等价
图边索引 `graphedge_index`	`torch.equal` 为真	完全一致

损失的逐位相同是可以预期的：切片求和的加法顺序与逐行累加完全一致。预测概率的 10^{-7} 量级差异来自 `index_select` 与逐对索引在底层 kernel 上的归约顺序不同，属正常的浮点非结合性，远小于任何有意义的阈值。

7.5 优化效果

表 15 优化前后对比（RTX 4050 Laptop, 6 GB）

配置	单文档耗时	峰值显存	2 折 × 1 轮快检
原实现	4.4 s	6 824 MB（溢出）	—
仅开梯度检查点	1.7 s	2 483 MB	—
补丁 + 梯度检查点	**0.26–0.36 s**	2 483 MB	35 min → 3.5 min

综合加速比为 **12–17 倍**（不同文档长度下略有差异），5 折 30 轮的总耗时由外推的 47 小时降到约 4.2 小时，使全量实验在单卡笔记本上成为可能。这一改造是本次复现能在课程周期内完成 5 折交叉验证与消融实验的直接前提。

8 讨论与局限

8.1 本复现的已知偏差

如实记录与原论文设置的差异，是复现研究的基本要求。本次复现存在四处偏差，按影响程度排序如下。

1. Stage 1 未按原方案使用 LLM 全量打标 原论文用 LLM 为 5 478 个正例事件对逐对标注支撑句，本文因平台免费额度耗尽，改用以“事件所在句”为核心的规则法。第 5 节的一致性对照表明该替代方案在精确率上几乎无损（0.985）、在 97.4% 的样本上被 LLM 答案完全包含，但召回只有 0.753。

按原方法的设计，这一偏差会通过 CGL 影响训练：正样本偏少、负样本偏多，对齐目标比原方案更“保守”。但如第 6.5 节所证，原实现的 CGL 没有梯度通路，因此在*当前代码*下这一偏差并不产生任何实际后果——它是一个尚未被激活的偏差。换言之，本文的伪标签工作只完成了前半步：把数据造出来并验证了它确实接近 LLM 的选句；还差后半步“让模型真正用上它”。这也是我们主张把 CGL 修复列在仓库问题清单首位的直接原因。

2. 原文不可得，无法逐表核对 CogECI 发表于 Elsevier 期刊，本次未能取得全文。因此本文的方法描述完全依据官方代码重建，表中“iLIF 的 62.5”是公开可查的参照值，而非 CogECI 原文的主表数字。若原文实际报告的是方向口径（三分类）结果，则本文的 0.6493 不可与之直接比较。第 2.5 节用三条证据判定默认配置为存在性口径，但这是*推断*，不是原文声明。

3. 单种子、单一划分 全部实验固定 `seed=209`。5 折之间的标准差（F_1 为 0.0261）反映的是折间差异，不是种子间方差。要给出更可信的置信区间，至少需要 3–5 个种子重复。

4. 未做方向口径实验 仓库同时提供了三分类的 pkl，本文只跑了二分类。方向口径的参照值是 iLIF 的 47.8，若时间允许，这是一个成本低、信息量大的补充实验。

8.2 主结果增益的来源

第 6.5 节证实 CGL 在原实现中没有梯度通路，因此**本文不能把表 9 相对 iLIF 的 +2.4 归因于 CGL**。剔除这一项之后，本文复现与 iLIF 的差异只剩两处结构改动：SGM（句子图模块）与 DSCRM（双流独立参数），二者都是梯度通路正常的模块。

分列结果与这一归因相符：增益几乎全部集中在句内（+6.2），跨句基本持平（−0.3）。SGM 的作用范围正是句内——它在同一句内的多个事件之间建边、刻画句法依存；而跨句的边仍由 iLIF 原有的“按预测置信度建边”机制给出，SGM 并不额外提供篇章级信息。DSCRM 把两流的参数解耦，避免占 85.4% 的跨句样本主导句内流的参数更新，其收益同样更容易落在句内指标上。因此“句内涨、跨句平”不是偶然的数字分解，而是这两个模块作用范围的直接投影。

需要强调，这是一种*与数据一致*的解释，而不是被实验分离出来的因果结论。要真正拆开 SGM 与 DSCRM 各自的贡献，需要分别去掉二者重跑；本文因时间限制未做，这是后续工作清单上的第一项。

8.3 关于 CGL 作用的推断（非实验结论）

在 CGL 无法反传的前提下，关于“它是否有用”的任何判断都只能是设计层面的推断。第 6.5 节的直通估计实验给出了初步答案：梯度接通后，E3 在前 12 轮里每一轮都落后于 E1，平均低 9.7 个点。受算力限制该对照未跑满 30 轮，但至少在有限预算内，CGL 的梯度是净负担而不是增益。下面三点用于解释这一现象，它们本身仍属推断，而非被独立验证的结论。

其一，当伪标签在 97% 以上的样本上等于“两个事件所在句”时，CGL 实际上只教模型一件事——把事件对表示拉向它自身所在的两句。这在句内场景几乎等价于“事件对表示应与承载它的句子语义一致”，是一个很强的归纳偏置；但跨句场景的真正证据常分布在两句之外（例如中间句交代了因果链），此时“拉向两个事件句”未必指向正确方向。

其二，伪标签的形态本身难以两全。表 7 中，规则 C（区间 ±1 扩窗）的召回最高（0.840）而 F_1 最低（0.657），精度只有 0.539：扩窗确实纳入了更多“正确答案”，但同时引入大量 LLM 并未选择的无关键接句。若 CGL 将来生效，这类噪声会以负样本的形式把表示推离真实支撑句——这正是本文最终选择精度优先的规则 D 的原因。

其三，也是更根本的一点：CGL 的目标是让事件对表示与句子表示在余弦空间中靠近，而因果判别的依据是事件对之间的差异。一个把事件对表示整体拉向若干句向量的辅助目标，能否为二分类边界提供有效信号，取决于两套表示是否共享有意义的语义空间。这需要实验回答，本文未能回答。

8.4 工程层面的反思

本项目在工程上投入的时间明显多于算法设计，原因值得记录：

1. **仓库不完整比算法难复现更常见。**缺失 `utils.py` 会直接报错，尚属“友好”；缺失伪标签 CSV 却静默回退全零，会在几十小时后给出一个“看起来正常”的低分结果，且很难定位。这类静默降级是复现中最危险的一类问题，因此本文专门写了训练前终检脚本。
2. **性能问题容易被误判为算力不足。**最初把 4.4 秒/文档归因于“6 GB 卡太小”，但实测 GPU 利用率只有 8%，说明瓶颈是调度而非算力。若不看利用率而直接下单租卡，成本会白花。
3. **性能改写必须伴随等价性验证。**5 处改写里有 2 处（删除死代码、改选边实现）一旦出错不会报错、只会悄悄改变结果。逐项比对梯度与选边结果，是唯一可靠的验收方式。

8.5 合规性

课程要求“检索库与提示不得包含测试标签或测试样本”。本文的实现满足该要求：伪标签按文档独立生成；5 折中测试折文档不参与梯度更新；所有 LLM 调用的提示词与原始响应均完整留痕（`raw_*.jsonl`），可逐条审计。需要如实说明的是，本文沿用了原论文的“label-informed”设定——提示词中告知 LLM “该事件对确有因果”——这是原方法的设计，不是本项目的额外引入。

9 复现指南

9.1 代码地址

本项目全部源代码、工具脚本、伪标签生成器与实验日志已发布在 GitHub：

https://github.com/jiusheng58/cogeci-reproduction

仓库不含数据集、BERT 权重与训练权重（体积原因），获取方式见下文。

训练好的 5 折权重（`checkpoints/E1_full_rulelabels/fold{1..5}_best.pt`，各约 535 MB，合计约 2.6 GB）因体积过大未随提交包提供，仍保留在本机。按下文给出的训练命令重新运行即可得到逐位相同的结果（固定 `seed=209`，单卡约 4.2 小时）；如评审需要可直接向作者索取。

9.2 目录结构

```
cogeci-reproduction/
|-- _repo/ 官方仓库（含本项目补丁）
| |-- main.py 训练主循环（+ 权重保存、日志路径修正）
| |-- parameter.py 超参（+ --grad_ckpt, --cgl_w, --cgl_mode）
| |-- utils.py 自 iLIF 补齐
| |-- model/
| | |-- model.py 网络定义（5 处性能补丁 + CGL 参数化 + STE 分支）
| | |-- _orig_model.py 原版备份（用于等价性验证）
| | `-- bert-base-uncased/ 需自行下载
| |-- data/
| | |-- database/exist/*.pkl 自 iLIF 补齐
| | `-- examples/llm/ 伪标签 CSV（本项目生成）
| `-- checkpoints/<实验名>/ 每折最优权重
|-- tools/
| |-- event_span.py 子词位置 -> 词级 span 反推
| |-- pseudo_from_rule.py 规则法伪标签生成
| |-- label_sentences.py LLM 打标（含断点续跑、关思考）
| |-- agree_rule_llm.py 规则 vs LLM 一致性对照
| |-- agree_by_reltype.py 按 intra/inter 拆分对照
```

```
| |-- make_ablation_table.py 从日志生成消融表
| |-- validate_csv.py 训练前伪标签终检
| |-- equiv_patch.py 性能补丁数值等价性验证
| |-- bench_vram.py 显存峰值实测
| `-- estimate_tokens.py 打标 token 消耗估算
|-- report/ LaTeX 报告源文件（本文件所在目录）
|-- run_ablation.sh 消融实验串行队列
`-- logs_*.txt 各实验完整训练日志
```

9.3 复现步骤

步骤 1：环境

```
python -m venv .venv
.venv/Scripts/pip install torch==2.6.0 --index-url https://download.pytorch.org/whl/cu126
.venv/Scripts/pip install torch_geometric==2.5.3 transformers==4.46.3 ^
    scikit-learn tqdm networkx openai
```

注意 transformers 必须为 4.x（5.x 已移除 AdamW）。

步骤 2：补齐缺失文件 从 iLIF 仓库取得 utils.py 与 data/database/exist/*.pkl，下载 bert-base-uncased 至 _repo/model/。

步骤 3：生成伪标签并终检

```
cd D:/wangbclass/cogeci
for S in dev train; do for V in strict loose; do
  ./.venv/Scripts/python.exe tools/pseudo_from_rule.py --split $S --variant $V \
     --out_csv _repo/data/examples/llm/rule/${S}_${V}.csv
done; done

./.venv/Scripts/python.exe tools/validate_csv.py \
   --pkl _repo/data/database/exist/EventStoryLine_bert_intra_and_inter_train_features.pkl \
   --csv _repo/data/examples/llm/llama3help_train_sen.csv
```

步骤 4：训练 必须在 _repo/ 目录下运行（process_esc_csr.py 使用相对路径，否则读不到伪标签会静默回退全零）。

```
cd _repo
../.venv/Scripts/python.exe -u main.py --dataset_type EventStoryLine \
  --cache_path ./data/database/exist --model_name ./model/bert-base-uncased \
  --no_of_classes 2 --t_lr 2e-5 --lr 1e-4 --w 0.6 --threshold 2 --epoch 30 \
  --batch_size 1 --beta_intra 0.3 --beta_inter 0.7 --max_iteration 9 \
  --min_iteration 2 --num_heads 4 --mlp_drop 0.4 --seed 209 --gamma 2.0 \
  --grad_ckpt --cgl_w 6.0 --method E1_full_rulelabels
```

消融实验只需改 `--cgl_w`：E0 取 0，并把 `--method` 改为 `E0_no_cgl`；E3 取 `--cgl_mode ste`（权重仍为 6），`--method` 改为 `E3_ste_termgrad`。再加 `--fold_start 1 --fold_end 1` 可只跑 Fold 1。

步骤 5：一致性对照（可选）

```
./.venv/Scripts/python.exe tools/agree_rule_llm.py --split dev
./.venv/Scripts/python.exe tools/agree_by_reltype.py --split dev
./.venv/Scripts/python.exe tools/equiv_patch.py # 等价性验证
```

9.4 硬件与耗时参考

全部实验在 RTX 4050 Laptop（6 GB）上完成，单折训练约 50 分钟，5 折合计约 4.2 小时。若显存 $\geq$ 16 GB，可去掉 `--grad_ckpt` 以获得更快的单步速度，数值结果不变。

参考文献

[1] Zefan Zhang, 等. CogECI: 面向文档级事件因果识别的一致性图学习. Knowledge-Based Systems (Elsevier). 官方代码仓库：https://github.com/zefanZhang-cn/CogECI. 说明：本次复现未能取得该文全文，方法描述依据官方代码重建。

[2] Cheng Liu, Wei Xiang, Bang Wang. Identifying while Learning for Document Event Causality Identification. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2405.20608.

[3] Tommaso Caselli, Piek Vossen. The Event StoryLine Corpus: A New Benchmark for Causal and Temporal Relation Extraction. In *Proceedings of the Events and Stories in the News Workshop*, ACL, 2017.

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT*, 2019.

[5] Yinhan Liu, Myle Ott, Naman Goyal, 等. RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692, 2019.

[6] Meiqi Chen, Yixin Cao, Kunquan Deng, Mukai Li, Kun Wang, Jing Shao, Yan Zhang. ERGO: Event Relational Graph Transformer for Document-level Event Causality Identification. In *Proceedings of the 29th International Conference on Computational Linguistics (COLING)*, pages 2118–2128, 2022.

[7] Changsen Yuan, Heyan Huang, Yixin Cao, Yonggang Wen. Discriminative Reasoning with Sparse Event Representation for Document-level Event-Event Relation Extraction. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 16222–16234, 2023.

A 附录

A.1 伪标签 CSV 的数据契约

`process_esc_csr.py` 中的 `get_sent_ground` 对格式有硬性要求，任一字段不符都会触发静默回退。列定义见表 16。

表 16 伪标签 CSV 的列定义

列号	列名	要求
0	`id`	行号，无实义
1	`doc_id`	**被读取**，须与 pkl 中的 `doc_id` 完全一致
2	`text`	原文，不参与计算
3	`sentences`	句子列表
4	`event_pair`	事件对，不参与计算
5	`node_event_id`	事件编号，不参与计算
6	`answer`	**被读取**，须为 `str(list[str])` 形式

第 6 列的格式最容易出错：它必须是一整篇文档所有事件对答案的列表的字符串表示，例如`"['1,2', 'None', '3']"`，其中每个元素是一对事件的答案——正例给句子编号（逗号分隔），负例给字符串`'None'`。一条 `doc_id` 对应一行，`answer` 列表的长度必须等于该文档的事件对数，否则标签矩阵与事件对数不匹配，训练会直接报错或（更糟）静默错位。

A.2 LLM 调用的提示词与响应样例

以下为 raw_dev_sen.jsonl 中的一条真实记录（文档 37_1，事件对 [3,1]，模型 qwen3.8-27b，共 27 句）。

系统提示

```
You are an expert with few words and will only answer questions accurately
```

用户提示（节选）

```
The input document is labeled with an ordinal number before each sentence,
e.g. (0,1...) , knowing that event: earthquake and event: injured in the 2
and 0 sentences are causally related, earthquake means [A powerful earthquake
in Indonesia's Aceh province flattened buildings and sparked landslides
Tuesday, killing at least five people and injuring dozens in a region
devastated by the quake-triggered tsunami of 2004.]. injured means[Five dead,
dozens injured ...]. identify which sentences from the document articulate
the causal relationship between events earthquake and injured. ...
```

原始响应

```
[0, 2]
```

两个事件分别位于第 2 句与第 0 句，LLM 返回的集合恰好是 $\{0, 2\}$——这正是第 5 节“LLM 含两个事件句 = 97.4%”这一统计的直观来源。规则法在该样本上给出一致答案，也是“精确率 0.985”的具体体现。

A.3 实验命令与日志对照

表 17 实验与其启动参数、日志文件

实验	关键参数	日志
E1 完整方法	--cgl_w 6.0	logs_E1_full.txt
E0 去 CGL	--cgl_w 0	logs_E0_no_cgl.txt
E3 梯度修复	--cgl_mode ste	logs_E3_ste_12ep.txt
快速检查	--k_fold 2 --epoch 1	logout/seed_209/

A.4 本文对原始代码的全部改动

为便于审阅，列出所有改动点。除标注“改动数值”者外，均不改变模型行为。

表 18 对官方仓库的改动清单

文件	类型	内容
model/model.py	性能	5 处向量化改写（表 13），已验证数值等价
model/model.py	功能	CGL 权重 w=6 改为读取 args.cgl_w（默认 6.0）
model/model.py	实验	新增 --cgl_mode ste 直通估计分支（默认 hard，与原实现一致）
model/model.py	修复	删除死代码 true_sent；修复 sent_ground 重复追加
model/_orig_model.py	新增	原版备份，供等价性验证
parameter.py	功能	新增 --grad_ckpt、--cgl_w、--cgl_mode、--fold_start/--fold_end
main.py	功能	每折按验证 F_1 保存权重至 checkpoints/<method>/
main.py	兼容	日志目录 seed:{s} → seed_{s}（Windows 不允许冒号）
utils.py	补齐	自 iLIF 取得
tools/*	新增	9 个工具脚本（见第 9 节目录树）